CP3566 · APPLIED JAVA PROGRAMMING · TERM PROJECT

Fraud Case-Management — Assignment Brief

Read this first. It is the single source of what to build, what to hand in, and how it is marked. Read it alongside the concepts guide.

0 At a glance

Weight	**35%** of final grade — 20% the working code, 10% the report, 5% the demo recording.
Work in	Individually or in pairs (max 2). In a pair, demo marks split if one partner cannot explain the shared code.
Due	**June 15, 2026**
Submit on D2L	your 3-page report and your demo recording, plus a link to your GitHub repository (5–10 commits). See §5.

1 The scenario

You work for a bank's fraud team. Money transfers flow in all day, and a few of them are suspicious. Your job is to build the back-office system that **screens every transaction against a set of rules**, **opens a case** when something looks wrong, and lets staff **review and resolve** each case — keeping a permanent record of every action.

You will build it on **Spring Boot** — the same stack as the in-class JPA & H2 work: a **Spring MVC REST API** for the web layer and **Spring Data JPA** (Hibernate over an embedded H2 database) for persistence. The `starter/` is a runnable Spring Boot project — you grow it.

2 What you are given

- **The concepts guide** (`CP3566_Concepts_Explained.html`) — every idea this project uses, in build order. Read §10 for the short version of this brief.
- **A runnable Spring Boot starter** (`starter/`) — the data layer is done for you (the `@Entity` classes and `JpaRepository` interfaces), plus a generator that builds the H2 database and fills it with realistic transactions, including planted fraud that trips every rule. You do *not* create test data by hand.
- **This brief** — the exact requirements, contract, and rubric.
- **The report template** (`STUDENT_REPORT_TEMPLATE.html`) — the 3-page report you fill in.

What is fixed for you: the rules (§4.3), the legal case moves (§4.4), and the REST contract (§4.5). You implement these — you do not redesign them.

3 Getting started

Build in this order — each step works before the next:

- **Run the starter** (`./mvnw spring-boot:run`) so you have a populated database and the data layer to work against.
- **Study the data layer you were given** — the `@Entity` classes and `JpaRepository` interfaces (§4.1, §4.2).
- **Rule engine** — prove it opens the right cases from the seeded data (§4.3).
- **REST endpoints** — wrap the logic in the web layer (§4.5).
- **Security and screen** last (§4.6, §4.7).

4 What to build — the specification

4.1 Layered architecture

Keep three layers separate; the separation is part of the grade. **Web** (`@RestController` — take the request, return JSON, no logic) → **Logic** (`@Service` — the rule engine and case workflow) → **Data** (`JpaRepository` classes — the *only* code that touches the database; Spring Data writes their SQL).

4.2 Database access

All database access goes through your **Spring Data JPA repositories** — Hibernate generates **parameterised** SQL, so user input is never glued into a query (safe from SQL injection by default). The repositories are the only classes that touch the database. The rule thresholds live in a `rules` table (a JPA entity) so an admin can change them without editing code. If you add a custom query, use a derived method or a bound `@Query` parameter — never string-concatenate input.

4.3 The rules

Rule	Fires when	Default
R1	A single amount is too large.	amount ≥ $10,000
R2	Too many transactions too fast on one account.	≥ 5 within 60 min
R3	Counterparty is on the watchlist.	any match
R4 `bonus`	Structuring — several amounts just under the limit.	≥ 3 of $9,000–$9,999 within 24h

A match raises an alert and opens a `NEW` case. **R1–R3 are required; R4 earns bonus marks.**

4.4 The case workflow (state machine)

A case is always in one state. **Only these moves are legal** — any other move must be rejected with **`409`**, and a user whose role cannot make a legal move gets **`403`**.

Move	From → To	Allowed role
pickup	NEW → REVIEWING	ANALYST
escalate	REVIEWING → ESCALATED	ANALYST
send-back	ESCALATED → REVIEWING	INVESTIGATOR
close-false	REVIEWING → CLOSED_FALSE	ANALYST
close-false	ESCALATED → CLOSED_FALSE	INVESTIGATOR
close-fraud	ESCALATED → CLOSED_FRAUD	INVESTIGATOR

4.5 The REST endpoints (the contract)

Build exactly these. Return **JSON** and the **correct status code**.

Method	Path	Who	Status codes
POST	`/api/login`	—	200, 401
GET	`/api/cases` [?status=]	any	200
GET	`/api/cases/{id}`	any	200, 404
POST	`/api/cases/{id}/pickup`	ANALYST	200, 403, 404, 409
POST	`/api/cases/{id}/escalate`	ANALYST	200, 403, 404, 409
POST	`/api/cases/{id}/send-back`	INVESTIGATOR	200, 403, 404, 409
POST	`/api/cases/{id}/close-false`	ANALYST / INVESTIGATOR	200, 403, 404, 409
POST	`/api/cases/{id}/close-fraud`	INVESTIGATOR	200, 403, 404, 409
POST	`/api/cases/{id}/notes`	any	201, 404
GET	`/api/rules`	any	200
PUT	`/api/rules/{code}`	ADMIN	200, 403, 404

4.6 Security

Two checks, on the server, for every protected action: **login first** (not logged in → **`401`**), then **role** (logged in but not allowed → **`403`**). Roles: `ANALYST`, `INVESTIGATOR`, `ADMIN`.

Users sign in with a **username and password**. The starter seeds the users with **hashed** passwords (BCrypt) — never store a password in plain text — so your login compares the submitted password against the stored hash; a wrong username *or* password returns **`401`**. The seeded demo logins are listed in the starter README.

4.7 The dashboard (screen)

A simple web page that lists the cases and lets a logged-in user open one and act on it. Use a server-side **Thymeleaf** template or a static HTML + JavaScript page that calls your REST API — your choice. Keep business logic out of the page.

4.8 The audit log

Every action (a case opened, picked up, escalated, closed, a note added) writes one permanent line: who, what, which case, when.

5 What to hand in

- **Your code in a GitHub repository** — builds and runs (`./mvnw spring-boot:run` against the H2 database the starter creates), implementing §4. The repo must show **5–10 commits** with meaningful messages (steady progress, not one giant commit) and must include your `PROMPTS.md` log (§7).
- **A 3-page plain-language report** — fill in `STUDENT_REPORT_TEMPLATE.html`: explain, with no code or jargon, how your project works.
- **A demo screen recording** (≈5–10 min) — run the app and walk one case from New to Closed, explaining each step *and* how your code works.

Due: June 15, 2026. On **D2L**, submit your **report** and your **demo recording** (the video file, or a link to it), plus a **link to your GitHub repository**. The repo must contain your code, your `PROMPTS.md`, and **5–10 commits** — a missing or single-commit history loses marks. The commit history together with `PROMPTS.md` is how we see the work grew over time.

6 How it is marked

Each row is marked **Full / Partial / Missing**. The marks equal your percentage of the course: **20 for the working code** (the first four rows), **10 for the report**, **5 for the demo** — 35 total, plus a 2-mark bonus.

Criterion	Full marks means	Marks
Data layer (Spring Data JPA)	JPA repositories are the only DB access; queries are parameterised (no string-built SQL).	5
Rule engine (R1–R3)	Seeded fraud opens the right cases; thresholds read from the DB.	5
Workflow & security	State machine enforced (409); password login + roles enforced (401/403); audit log written.	5
REST endpoints & status codes	The §4.5 endpoints return correct JSON and status codes.	5
Project report	The 3-page plain-language report explains the system clearly.	10
Demo recording	Screen recording shows the app working end-to-end, with you explaining the code.	5
R4 structuring bonus	Structuring rule implemented and demonstrated.	+2

PROMPTS.md gate: the four work criteria above (20 marks) are awarded only if a genuine `PROMPTS.md` log is submitted. A **missing or fabricated log makes those 20 marks zero**, regardless of how good the code is. The report (10) and demo (5) are marked separately. Using any AI other than GitHub Copilot zeroes the *whole* project (§7).

7 Using AI tools & academic integrity

> **Read this carefully — it is strictly enforced.**

GitHub Copilot (and Copilot Chat) is the only AI assistant you may use. Copilot is permitted because it works inside the IDE and leaves an auditable trail. Using **any other AI tool** — ChatGPT, Claude, Gemini, Cursor, or anything similar — is an **automatic zero on the entire project**: no warnings, no partial credit.

Keep a `PROMPTS.md` log. Record every significant prompt you gave Copilot and, in a line or two, what you accepted or changed. Submit it with your code. A **missing or fabricated log zeroes the work marks** (the four §6 criteria — 20 marks); see the gate note in §6.

Your demo recording must show you explaining your code, not just clicking through the app. Walk through how the key parts work and why you wrote them that way. You may also be asked follow-up questions; if you cannot explain your code, you lose marks. For pairs, both partners should narrate, and demo marks are split if one cannot explain the shared code.
